

## MCP: Questions and Answers

**Q1. Explain the difference between a tool, a resource, and a prompt in MCP, with a one-line example of when you would reach for each.**

**Ans:** The difference is who is in control of each one.

- **Tool:** something Claude can *do*. It is a function the model decides to call, and you can use it for example, when you want Claude to generate a story and store it in database.
- **Resource:** something Claude can *read*. It is data or content the server exposes, such as a file, a database record, or a document, and it is usually chosen by the user. Example: use a resource to let Claude read your project's `README`.
- **Prompt:** a *reusable template* for starting a task. It is a pre-written instruction the user picks. Example: use a prompt for a “review this pull request” template that appears as a shortcut in the chat.

In short: tools are model-controlled, resources are app-controlled, and prompts are user-controlled.

**Q2. A local MCP server usually talks to Claude Desktop over `stdio` instead of a normal HTTP API. What does that mean in practice, and why does it make sense for this use case?**

**Ans:** `stdio` means “standard input and output,” the same pipes any command-line program uses. Claude Desktop launches the MCP server as a child process on your own computer. It sends messages by writing to the server's `stdin` and reads replies from its `stdout`. The messages are JSON. There is no port, no URL, and no web server, and when Claude Desktop closes, it stops the process.

It makes sense for local use because:

- no web server, port, or HTTP routes to set up.
- it runs as your own user on your own machine so no API keys or login flows are required

## FastMCP Documentation Questions

**Q3. FastMCP supports three transports: stdio, http, and sse. Read the deployment docs and explain when you'd pick stdio vs http, and what actually changes in your mcp.run() call to switch between them.**

**Ans:** stdio is default transport for mcp servers. When you call run() without arguments, your server uses STDIO transport. STDIO is ideal for: Local development and testing, Claude Desktop integration, Command-line tools, Single-user applications

Unlike STDIO where each client gets its own process, an HTTP server can handle multiple clients simultaneously. The Streamable HTTP protocol provides full bidirectional communication between client and server, supporting all MCP operations including streaming responses.

### **What changes in mcp.run()**

Only the arguments change. tools and the rest of the server stay the same.

stdio (default): no arguments

mcp.run()

http: pick the transport and the network binding

mcp.run(transport="http", host="127.0.0.1", port=8000)

**Q4. FastMCP automatically turns some tool return values into "structured output" and others into plain text. Read the tools docs and explain the difference between what happens when a tool returns a plain string vs when it returns a dict or a Pydantic model, and why that distinction would matter to a client calling your tool.**

**Ans:** what happens when tool returns a plain string

Plain string: the client gets just text. FastMCP sends it as a text block. It is fine for a human or an LLM to read if your function has '-> str' it's still only one piece of text.

Dict or Pydantic model: FastMCP sends the normal text version and also structured output, which is real JSON with named fields. With a Pydantic model it also publishes an output schema, so the client knows the fields and

types in advance.

Why it matters to the client: - With a string like "Story saved: Eiffel-Tower.md", the client has to parse the text to get the filename, and that breaks if the wording changes.

- With something like "filename": "Eiffel-Tower.md", "saved": true, the client reads `result["filename"]` directly. So plain strings suit tools meant for an LLM or a person to read and dicts or models suit tools whose results other code will use.